BÀI TẬP TÌM KIẾM TRONG AI

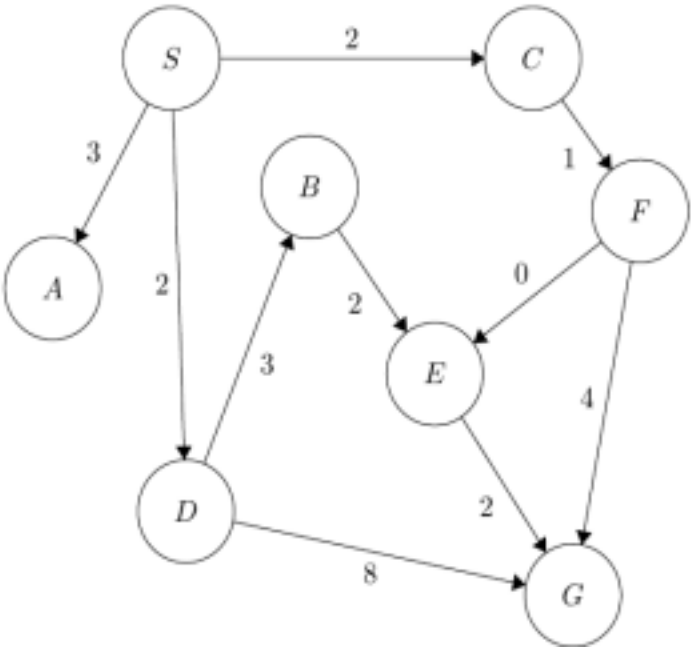
Họ và tên	Trương Quốc Lê Huy
MSSV / Lớp	3123410132

Mục tiêu học tập

• Mô phỏng các thuật toán BFS, DFS và Iterative Deepening.
• Biểu diễn bài toán Tháp Hà Nội dưới dạng bài toán tìm kiếm.
• Thiết kế và phân tích heuristic cho A*.
• So sánh DFS, BFS, UCS, Greedy Search và A* trên cùng một bài toán.
• Phân tích các phát biểu lý thuyết về heuristic, A*, Minimax và Expectimax.

Bài 1. Các thuật toán tìm kiếm

Cho đồ thị có trạng thái bắt đầu S và trạng thái đích G như hình dưới đây.



Hình 1. Đồ thị dùng cho các thuật toán tìm kiếm

Yêu cầu: Với mỗi thuật toán, hãy ghi thứ tự các nút được đưa vào explored set, đường đi được trả về và tổng chi phí của đường đi. Nếu có hòa, phá hòa theo thứ tự alphabet của trạng thái cuối cùng trong đường đi.

1.1. Breadth-First Search (BFS)

Nội dung	Kết quả
Explored Set	S, A, C, D, F, B
Path	S → D → G

Total Cost	2+8=10

Giải quyết vấn đề bằng tìm kiếm **2** / **8**

1.2. Depth-First Search (DFS)

Nội dung	Kết quả
Explored Set	S, A, C, F, E
Path	S → C → F → E → G
Total Cost	$2 + 1 + 0 + 2 = 5$

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G
...	

Path cuối cùng: S → D → G

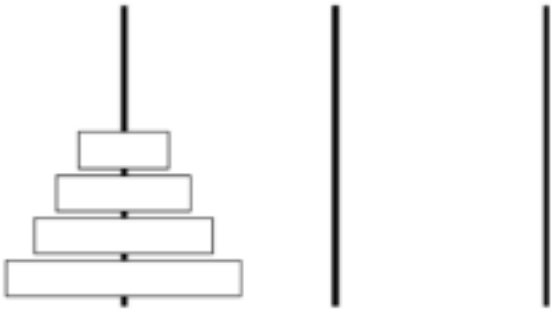
Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

Trả lời: BFS dùng hàng đợi (Queue) ưu tiên duyệt theo chiều rộng, tìm được đường đi có số cạnh ít nhất (ngắn nhất về số bước). DFS dùng ngăn xếp (Stack) ưu tiên duyệt theo chiều sâu, có thể tìm thấy đích nhanh chóng nhưng đường đi thường không tối ưu về số bước cũng như chi phí

Bài 2. Tháp Hà Nội - Tower of Hanoi

Bài toán gồm n đĩa có kích thước khác nhau và ba cọc. Ban đầu các đĩa được xếp trên một cọc theo thứ tự từ lớn đến nhỏ. Mỗi lần chỉ được di chuyển một đĩa và không được đặt đĩa lớn lên trên đĩa nhỏ. Mục tiêu là chuyển toàn bộ đĩa sang cọc thứ ba.



Hình 2. Trạng thái ban đầu của bài toán Tháp Hà Nội

2.1. Biểu diễn trạng thái

Hãy đề xuất cách biểu diễn một trạng thái của bài toán. Biểu diễn phải cho biết mỗi đĩa đang ở cọc nào và đủ để xác định các hành động hợp lệ.

Trả lời: Một trạng thái có thể được biểu diễn bằng một tuple (danh sách) gồm n phần tử, $(c_1, c_2, \ldots, c_n)$, trong đó $c_i \in \{1, 2, 3\}$ biểu thị số thứ tự của cọc chứa đĩa thứ i (đĩa được đánh số từ nhỏ nhất đến lớn nhất)

_ 2.2. Kích thước không gian trạng thái

Với n đĩa, hãy xác định số trạng thái có thể có theo cách biểu diễn của bạn và giải thích cách suy luận.

Trả lời: Mỗi đĩa trong n đĩa có thể nằm ở 1 trong 3 cọc. Do đó, có $3 \times 3 \times \ldots \times 3 = 3^n$ trạng thái có thể

Giải quyết vấn đề bằng tìm kiếm **3** / **8**

_ 2.3. Trạng thái bắt đầu

Hãy viết trạng thái khởi đầu S0 theo cách biểu diễn đã chọn.

Trả lời: Tất cả đĩa đều ở cọc 1 (cọc ngoài cùng bên trái), nên $S_0 = (1, 1, \ldots, 1)$

_ 2.4. Các hành động hợp lệ

Từ một trạng thái bất kỳ, hãy mô tả tập Actions(s). Một hành động hợp lệ phải thỏa mãn các điều kiện nào?

Trả lời: Từ trạng thái hiện tại, có thể di chuyển đĩa nằm trên cùng của một cọc sang cọc khác. Điều kiện hợp lệ:

1. Cọc gốc phải có ít nhất 1 đĩa.
2. Đĩa được di chuyển (đĩa trên cùng cọc gốc) phải nhỏ hơn đĩa đang nằm trên cùng của cọc đích, hoặc cọc đích đang rỗng

__ 2.5. Goal Test

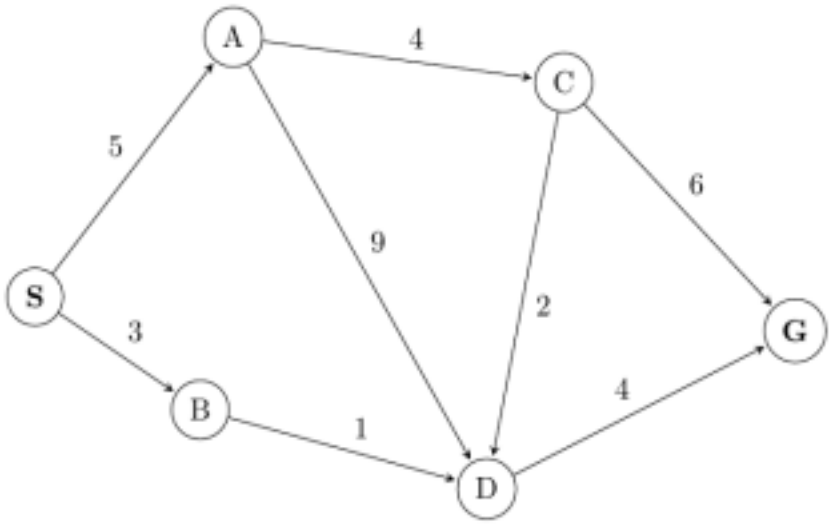
Hãy mô tả hàm goalTest(state): khi nào hàm trả về True?

Trả lời: Hàm goalTest(state) trả về True khi tất cả các đĩa đều nằm ở cọc thứ 3, tức là trạng thái hiện tại là $(3, 3, \ldots, 3)$

Bài 3. Thiết kế và phân tích Heuristic

Xét đồ thị dưới đây. Hãy thiết kế heuristic và phân tích ảnh hưởng của heuristic đến thuật toán

A*.



Hình 3. Đồ thị dùng để thiết kế heuristic

Công thức A*: f(n) = g(n) + h(n), trong đó g(n) là chi phí thực tế từ S đến n và h(n) là chi phí ước lượng từ n đến đích.

3.1. Thiết kế heuristic admissible và consistent

Điền giá trị h(n) cho từng nút sao cho heuristic phù hợp với yêu cầu của nhóm.

3

Giải quyết vấn đề bằng tìm kiếm **4** / **8**
Chọn $h(n)$ bằng chính chi phí thực tế $g(n)$

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

3.2. Chạy A* với heuristic đã thiết kế

Explored Set: S, B, D

Path: S → B → D → G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Đề xuất một bộ giá trị h(n) admissible nhưng không consistent. Chỉ ra ít nhất một cạnh vi phạm điều kiện h(n) <= c(n,n') + h(n').

Trả lời: Giữ nguyên $h(G) = 0$. Chọn một cạnh, ví dụ $B \to D(cost = 1)$. Ta cần vi phạm $h(B) \leq cost(B, D) + h(D)$.

Đặt $h(D) = 1$.

Cần $h(B) > 1 + 1 = 2$. Đặt $h(B) = 4$.

Kiểm tra tính admissible: $h(D) = 1 \leq h^*(D) = 4$ (Thỏa), $h(B) = 4 \leq h^*(B) = 5$ (Thỏa).

Bộ heuristic đề xuất: $h(G) = 0, h(D) = 1, h(C) = 6, h(B) = 4, h(A) = 10, h(S) = 8$.

Cạnh vi phạm: Cạnh $B \to D$ vì $h(B) = 4 > cost(B, D) + h(D) = 1 + 1 = 2$.

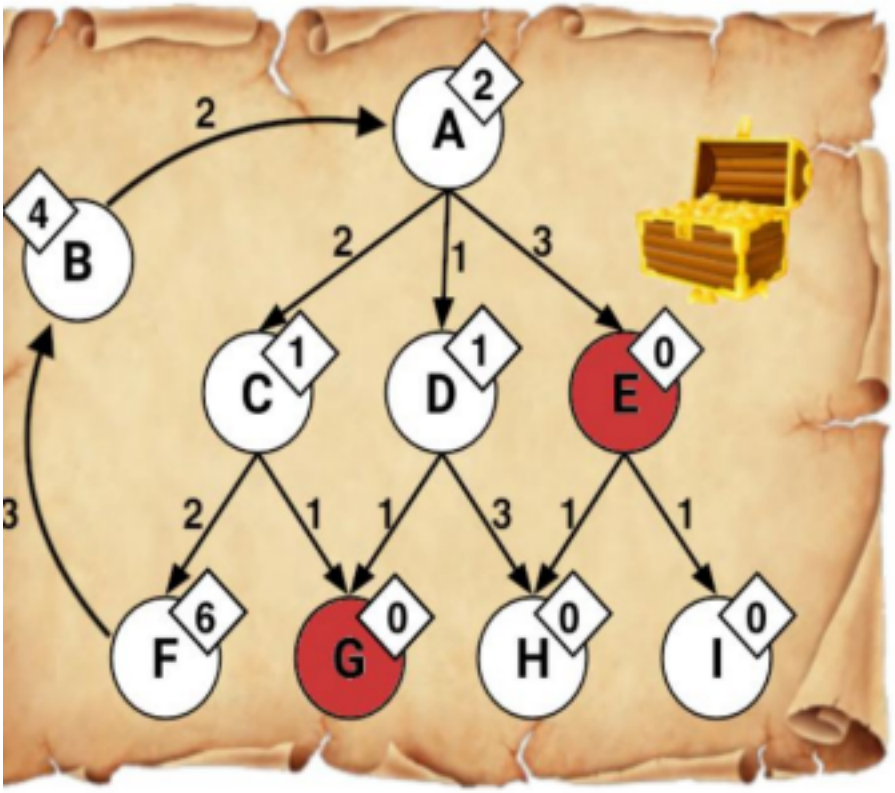
_ 3.4. Câu hỏi mở rộng

Giải thích vì sao một heuristic consistent thì cũng admissible, giả sử h(G) = 0.

Trả lời: Giả sử $h(n)$ là consistent, tức là $h(n) \leq cost(n, m) + h(m)$ với mọi cạnh (n, m). Xét một đường đi bất kỳ từ n đến đích G: $n \to n_1 \to n_2 \to \ldots \to n_k \to G$. Ta có: $h(n) \leq cost(n, n_1) + h(n_1)$ $h(n_1) \leq cost(n_1, n_2) + h(n_2)$ ... $h(n_k) \leq cost(n_k, G) + h(G)$ Cộng vế theo vế các bất đẳng thức trên: $h(n) \leq cost(n, n_1) + cost(n_1, n_2) + \ldots + cost(n_k, G) + h(G)$ Vì $h(G) = 0$, ta có $h(n) \leq$ (tổng chi phí đường đi từ n đến G). Do điều này đúng với mọi đường đi đến G, nó phải đúng với đường đi tối ưu, tức là $h(n) \leq h^*(n)$. Vậy heuristic là admissible

Bài 4. Treasure Hunting (Tìm kho báu)

A là đảo xuất phát. Các nút màu đỏ là các đảo có kho báu; chỉ cần tới một trong các đảo đó. Mũi tên biểu diễn hành động có hướng; số trên cạnh là action cost. Giá trị trong hình thoi là heuristic ước lượng chi phí tối thiểu từ nút hiện tại tới một kho báu.



Hình 4. Bản đồ tìm kho báu (giữ nguyên hình gốc)

Quy tắc phá hòa: khi lấy phần tử khỏi frontier, nếu hòa thì chọn theo thứ tự alphabet.

4.1. Depth-First Search

Explored Set: A, C, F, B

_ Path returned: A → C → G

_ 4.2. Breadth-First Search

Explored Set: A, C, D

_ Path returned: A → E

Giải quyết vấn đề bằng tìm kiếm **6** / **8**

4.3. Uniform-Cost Search

Explored Set: A → E

_ Path returned: A → D → G

_ Cost: 2

_ 4.4. Greedy Search

Explored Set: A

__ Path returned: A → E

Cost: 3

4.5. A* Search

Explored Set: A, D

_ **Path returned:** A → D → G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu	$A \rightarrow C \rightarrow G$	3

Giải quyết vấn đề bằng tìm kiếm **7** / **8**

BFS	Độ sâu nhỏ nhất	$A \rightarrow E$	3
UCS	g(n)	$A \rightarrow D \rightarrow G$	2
Greedy	h(n)	$A \rightarrow E$	3
A*	g(n)+h(n)	$A \rightarrow D \rightarrow G$	2

Bài 5. Câu hỏi Đúng/Sai

Với mỗi phát biểu, hãy ghi Đúng hoặc Sai và giải thích ngắn gọn. Nếu sai, nên đưa ra phản ví dụ.

Câu 1. Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic. Đúng / Sai: _____Sai_____

Giải thích: DFS không quan tâm đến chi phí, nó có thể vô tình đi thẳng một mạch xuống nhánh chứa trạng thái đích ngay từ đầu và tìm thấy đích với số lượng node mở rộng rất ít. Trong khi đó, A* phải mở rộng tất cả các node có chi phí $f(n)$ nhỏ hơn chi phí tối ưu, do đó trong một số trường hợp (đặc biệt khi DFS may mắn), DFS có thể mở rộng ít node hơn A*

Câu 2. Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B. Đúng / Sai: _____Sai_____

Giải thích: Heuristic admissible yêu cầu $h(n)$ không bao giờ được lớn hơn chi phí thực tế. Một quân xe có thể di chuyển ngang 5 ô chỉ trong 1 nước đi (chi phí thực tế = 1), nhưng khoảng cách Manhattan sẽ tính là 5 ô (ước lượng = 5). Vì ước lượng lớn hơn chi phí thực tế ($5 > 1$), Manhattan không phải là heuristic admissible trong trường hợp này

Câu 3. Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman. Đúng / Sai: _____Đúng_____

Giải thích: Khoảng cách Euclidean (khoảng cách đường thẳng) luôn là khoảng cách ngắn nhất giữa hai điểm trong không gian. Vì Pacman phải đi vòng qua các bức tường trên không gian lưới, quãng đường thực tế luôn lớn

hơn hoặc bằng khoảng cách đường thẳng. Do đó, $h(n) \leq h^*(n)$, thỏa mãn điều kiện admissible

Câu 4. Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

Đúng / Sai: Sai

Giải thích: Giả sử có hai heuristic admissible là $h_1(n)$ và $h_2(n)$ (nghĩa là $h_1 \leq h^*$ và $h_2 \leq h^*$). Khi cộng lại, tổng $h_1(n) + h_2(n)$ hoàn toàn có thể lớn hơn chi phí thực tế $h^*(n)$, làm mất đi tính admissible. Để kết hợp nhiều heuristic admissible, ta dùng hàm Max: $h(n) = \max(h_1(n), h_2(n))$ thay vì tính tổng

Câu 5. Admissibility của một heuristic cho A* kéo theo consistency.

Đúng / Sai: Sai

Giải thích: Chiều ngược lại mới đúng: Tính nhất quán (consistency) là điều kiện mạnh hơn và luôn kéo theo tính admissible. Một heuristic có thể admissible (không đánh giá cao hơn chi phí thực đến đích) nhưng vẫn có thể không consistent (vi phạm bất đẳng thức tam giác giữa các node kề nhau

Câu 6. A* với graph search luôn tối ưu nếu heuristic admissible.

Đúng / Sai: Sai

Giải thích: Để A* với Graph Search đảm bảo tính tối ưu, heuristic bắt buộc phải **consistent** (nhất quán). Nếu heuristic chỉ admissible mà không consistent, Graph Search có thể đưa một trạng thái vào Explored set thông qua một đường đi chưa tối ưu và sẽ bỏ lỡ đường đi tối ưu thực sự nếu không có cơ chế mở lại node. (Admissible chỉ đủ để đảm bảo tối ưu cho Tree Search).

Tổng kết nhanh

Nhóm	Thuật toán	Hàm/tiêu chí chính
Uninformed Search	BFS / DFS / IDS	Không sử dụng heuristic
Cost-based Search	UCS	g(n)
Informed Search	Greedy	h(n)

Giải quyết vấn đề bằng tìm kiếm **8** / **8**

Informed Search	A*	g(n)+h(n)

Ghi nhớ: A* dùng f(n) = g(n) + h(n). ---o0o---

(Hết)8

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Mục tiêu học tập

• Mô hình hóa bài toán tìm kiếm trong AI.

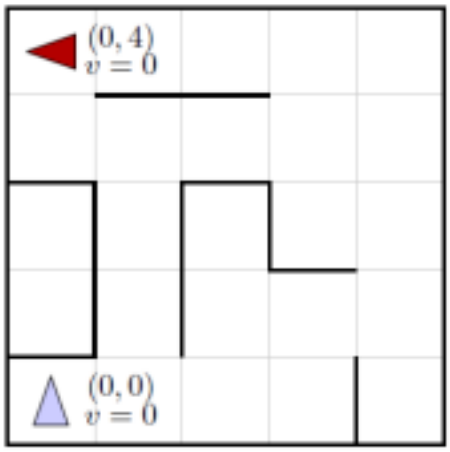
• Vận dụng BFS, UCS, Greedy và A*.

• Phân tích và đánh giá heuristic.

• Phân biệt heuristic admissible và consistent.

• So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

• Tăng tốc (Faster): v' = v + 1

• Giảm tốc (Slower): v' = v - 1

• Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

• Rẽ trái: quay 90° ngược chiều kim đồng hồ.

• Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

• Hành động làm tác tử va vào tường.

• Làm v < 0 hoặc v > Vmax.

• Giữ tốc độ khi v = 0.

• Rẽ khi v ≠ 0.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn (v = 0) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

1. Tăng tốc (v=1) → tiến đến (0,1).
2. Giữ tốc độ (v=1) → tiến đến (0,2).
3. Giảm tốc (v=0) → dừng tại (0,2).
4. Rẽ phải (hướng Đông).
5. Tăng tốc (v=1) → tiến đến (1,2) (thỏa mãn đi qua nhánh ngoài cùng bên trái).
6. Giảm tốc (v=0) → dừng tại (1,2).
7. Rẽ trái (hướng Bắc).
8. Tăng tốc (v=1) → tiến đến (1,3).
9. Giữ tốc độ (v=1) → tiến đến (1,4).
10. Giảm tốc (v=0) → dừng tại (1,4).
11. Rẽ trái (hướng Tây).
12. Tăng tốc (v=1) → tiến đến (0,4).
13. Giảm tốc (v=0) → dừng hẳn tại (0,4) hướng Tây

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

$= M \times N \times 4 \times (Vmax + 1)$. (Bao gồm $M \times N$ tọa độ, 4 hướng quay, và các mức vận tốc từ 0 đến $Vmax$)

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

■ Có ☐ Không

Ví dụ: Khi tác tử đang có vận tốc $v \geq 1$, đối mặt với bức tường ngay trước mặt (cách 1 ô). Tác tử không thể rẽ (do $v \neq 0$), và mọi hành động (Tăng, Giảm, Giữ) đều làm tác tử tiến tới và đâm vào tường, tạo ra hành động không hợp lệ

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Là 3. Khi $v = 0$, có 3 hành động hợp lệ: Tăng tốc, Rẽ trái, Rẽ phải. Khi $0 < v < Vmax$, có 3 hành động hợp lệ: Tăng tốc, Giảm tốc, Giữ tốc độ

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ■ Không

Phản ví dụ: Quãng đường cần đi là 6 ô trên một đường thẳng. Khoảng cách Manhattan là 6. Tuy nhiên, nếu Vmax = 3, tác tử có thể tăng tốc (v=1, 2, 3) và đi hết 6 ô chỉ với ít hơn 6 hành động. Do chi phí ước lượng (6) lớn hơn chi phí thực tế (số hành động), nó không admissible

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

■ Có ☐ Không

Số bước di chuyển thực tế (hành động) nhỏ nhất để đi hết một quãng đường luôn lớn hơn hoặc bằng quãng đường đó chia cho vận tốc lớn nhất có thể. Do đó, ước lượng này không bao giờ vượt quá số hành động thực tế

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ■ Không

Thuật toán A* Tree Search vẫn sẽ vét cạn đồ thị hữu hạn và tìm thấy đích, chỉ là nó có thể mở rộng nhiều node hơn mức cần thiết

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

■ Có ☐ Không

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

■ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

■ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức: $d(n) \leq d(n_s)$

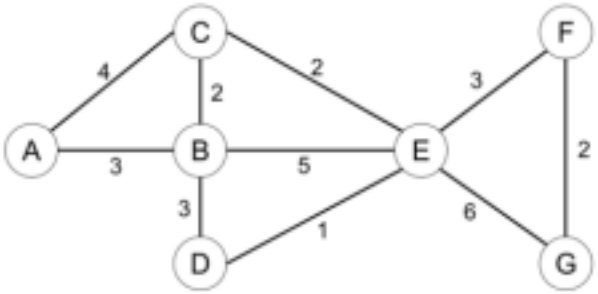
b) Uniform-Cost Search (UCS).

Bất đẳng thức: $g(n) \leq g(n_c)$

c) A* Search, giả sử h là heuristic nhất quán.

Bất đẳng thức: $g(n) + h(n) \leq g(n_c)$

Câu 3. Tìm kiếm trên đồ thị



Node	h_1	h_2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4

E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	■	☐	☐	☐
Greedy với h1	☐	☐	☐	■
Greedy với h2	■	☐	☐	☐
A* với h1	☐	☐	☐	■
A* với h2	■	☐	☐	☐

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 7

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ■ Không Consistent: ☐ Có ■ Không

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ■ Có ☐ Không Consistent: ■ Có ☐ Không

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

☐ Đúng ■ Sai

Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.

■ Đúng ☐ Sai

Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete. ■ Đúng ☐ Sai

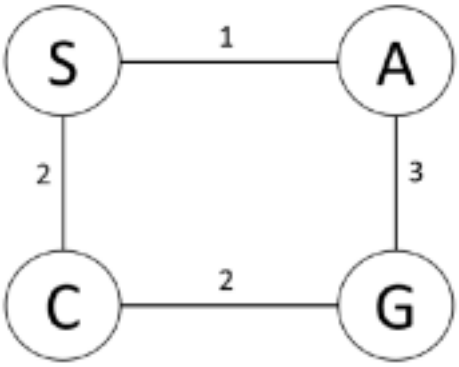
Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.

☐ Đúng ■ Sai

Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS. ☐ Đúng ■ Sai

Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent. ■ Đúng ☐ Sai

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Heuristic Function	
State	$h(s)$
S	
A	
C	
G	0

$h(S)$: 3 (Tổng chi phí thực từ S đến G là $1 + 2 = 3$. Đặt h(S)=3 để admissible).
$h(A)$: 1 (Chi phí thực là 2. Đặt h(A)=1 để admissible).
$h(C)$: 2 (Giả định nhánh C cũng hợp lệ và admissible).
$h(G)$: 0 *Kiểm tra:* $h(S) = 3 > cost(S, A) + h(A) = 1 + 1 = 2$ → Vi phạm consistent tại cạnh S-A, nhưng vẫn admissible

--- Hết ---